

AFRILANG Stdlib

std/c/finrisk

Français. Bibliothèque AFRILANG 'std/c/finrisk' (niveau complex, catégorie complex). Elle expose 7 fonction(s) : riskSum, riskMean, riskMin, riskMax, riskVar, riskStd, riskNorm.

```
'import "std/c/finrisk"' · 'use finrisk'
```

```
- 'riskSum(v list number) → number' - 'riskMean(v list number) → number' - 'riskMin(v list number) → number' - 'riskMax(v list number) → number' - 'riskVar(v list number) → number' - 'riskStd(v list number) → number' - 'riskNorm(v list number) → list number'
```

English.

AFRILANG library 'std/c/finrisk' (tier complex, category complex). Exports 7 function(s): riskSum, riskMean, riskMin, riskMax, riskVar, riskStd, riskNorm.

```
'import "std/c/finrisk"' · 'use finrisk'
```

```
- 'riskSum(v list number) → number' - 'riskMean(v list number) → number' - 'riskMin(v list number) → number' - 'riskMax(v list number) → number' - 'riskVar(v list number) → number' - 'riskStd(v list number) → number' - 'riskNorm(v list number) → list number'
```

Catégorie / Category	Modules complex (c/)	
Niveau / Tier	complex	June 16, 2026
Fonctions / Functions	7	

Contents

Français

1. Vue d'ensemble

Bibliothèque AFRILANG 'std/c/finrisk' (niveau complexe, catégorie complexe). Elle expose 7 fonction(s) : riskSum, riskMean, riskMin, riskMax, riskVar, riskStd, riskNorm.

'import "std/c/finrisk"' · 'use finrisk'

```
- `riskSum(v list number) → number`
- `riskMean(v list number) → number`
- `riskMin(v list number) → number`
- `riskMax(v list number) → number`
- `riskVar(v list number) → number`
- `riskStd(v list number) → number`
- `riskNorm(v list number) → list number`
```

2. Installation & import

Ajoutez le module à votre programme :

```
import "std/c/finrisk"
use finrisk
```

Niveau : complexe · Catégorie : Modules complexe (c/)
Domaines avancés – graphes, NLP, réseaux complexes.

3. Référence API

- riskSum(v list number) → number•
riskMean(v list number) → number•
riskMin(v list number) → number•
riskMax(v list number) → number•
riskVar(v list number) → number•
riskStd(v list number) → number•
riskNorm(v list number) → list

4. Exemples d'utilisation

```
import "std/c/finrisk"
use finrisk
```

```
create result = riskSum(/* args */)
say result
```

```
create result = riskMean(/* args */)
say result
```

```
create result = riskMin(/* args */)
say result
```

```
create result = riskMax(/* args */)
say result
```

```
create result = riskVar(/* args */)
say result
```

```
create result = riskStd(/* args */)
say result
```

5. Bonnes pratiques

- Préférez ``import "std/c/finrisk"`` en tête de fichier.
- Lancez ``afrilang check`` avant de livrer.
- Combinez avec les modules stdlib de la même catégorie.
- Voir `/stdlib/` pour le source live et les démos playground.
- Signalez les problèmes sur GitHub si une export manque.

6. Annexe — source

module finrisk

```
function riskSum(v list number) returns number
end
```

```
function riskMean(v list number) returns number
end
```

```
function riskMin(v list number) returns number
end
```

```
function riskMax(v list number) returns number
end
```

```
function riskVar(v list number) returns number
end
```

```
function riskStd(v list number) returns number
end
```

```
function riskNorm(v list number) returns list number
end
end
```

English

1. Overview

AFRILANG library 'std/c/finrisk' (tier complex, category complex). Exports 7 function(s): riskSum, riskMean, riskMin, riskMax, riskVar, riskStd, riskNorm.

'import "std/c/finrisk"' · 'use finrisk'

```
- `riskSum(v list number) → number`
- `riskMean(v list number) → number`
- `riskMin(v list number) → number`
- `riskMax(v list number) → number`
- `riskVar(v list number) → number`
- `riskStd(v list number) → number`
- `riskNorm(v list number) → list number`
```

2. Installation & import

Add the module to your program:

```
import "std/c/finrisk"
use finrisk
```

Tier: complex · Category: Complex modules (c/)
Advanced domains – graphs, NLP, complex networks.

3. API reference

- - riskSum(v list number) → number
 - riskMean(v list number) → number
 - riskMin(v list number) → number
 - riskMax(v list number) → number
 - riskVar(v list number) → number
 - riskStd(v list number) → number
 - riskNorm(v list number) → list

4. Usage examples

```
import "std/c/finrisk"
use finrisk
```

```
create result = riskSum(/* args */)
say result
```

```
create result = riskMean(/* args */)
say result
```

```
create result = riskMin(/* args */)
say result
```

```
create result = riskMax(/* args */)
say result
```

```
create result = riskVar(/* args */)
say result
```

```
create result = riskStd(/* args */)
say result
```

5. Best practices

- Prefer ``import "std/c/finrisk"`` at the top of your file.
- Run ``afrilang check`` before shipping.
- Combine with related stdlib modules from the same category.
- See `/stdlib/` for the live source and playground demos.
- Report issues on GitHub if an export is missing or undocumented.

6. Appendix — source

module finrisk

```
function riskSum(v list number) returns number
end
```

```
function riskMean(v list number) returns number
end
```

```
function riskMin(v list number) returns number
end
```

```
function riskMax(v list number) returns number
end
```

```
function riskVar(v list number) returns number
end
```

```
function riskStd(v list number) returns number
end
```

```
function riskNorm(v list number) returns list number
end
end
```

Référence rapide / Quick reference

- Import : `import "std/c/finrisk"`
- Vérification : `afrilang check`
- Documentation : <https://afrilang.dev/stdlib/std/c/finrisk/>

Source (extrait)

```
module finrisk

function riskSum(v list number) returns number
end

function riskMean(v list number) returns number
```

```
end

function riskMin(v list number) returns number
end

function riskMax(v list number) returns number
end

function riskVar(v list number) returns number
end

function riskStd(v list number) returns number
end

function riskNorm(v list number) returns list number
end
end
```